

Gebze Technical University
Department of Computer Engineering

CSE 344 – System Programming
Term Project Report

Concurrent Stock Exchange Simulator

Name Surname: Ayşe Feyza SERBEST

Student ID: 220104004052

Course: CSE 344 – System Programming

Semester: Spring 2025

Date: June 5, 2026

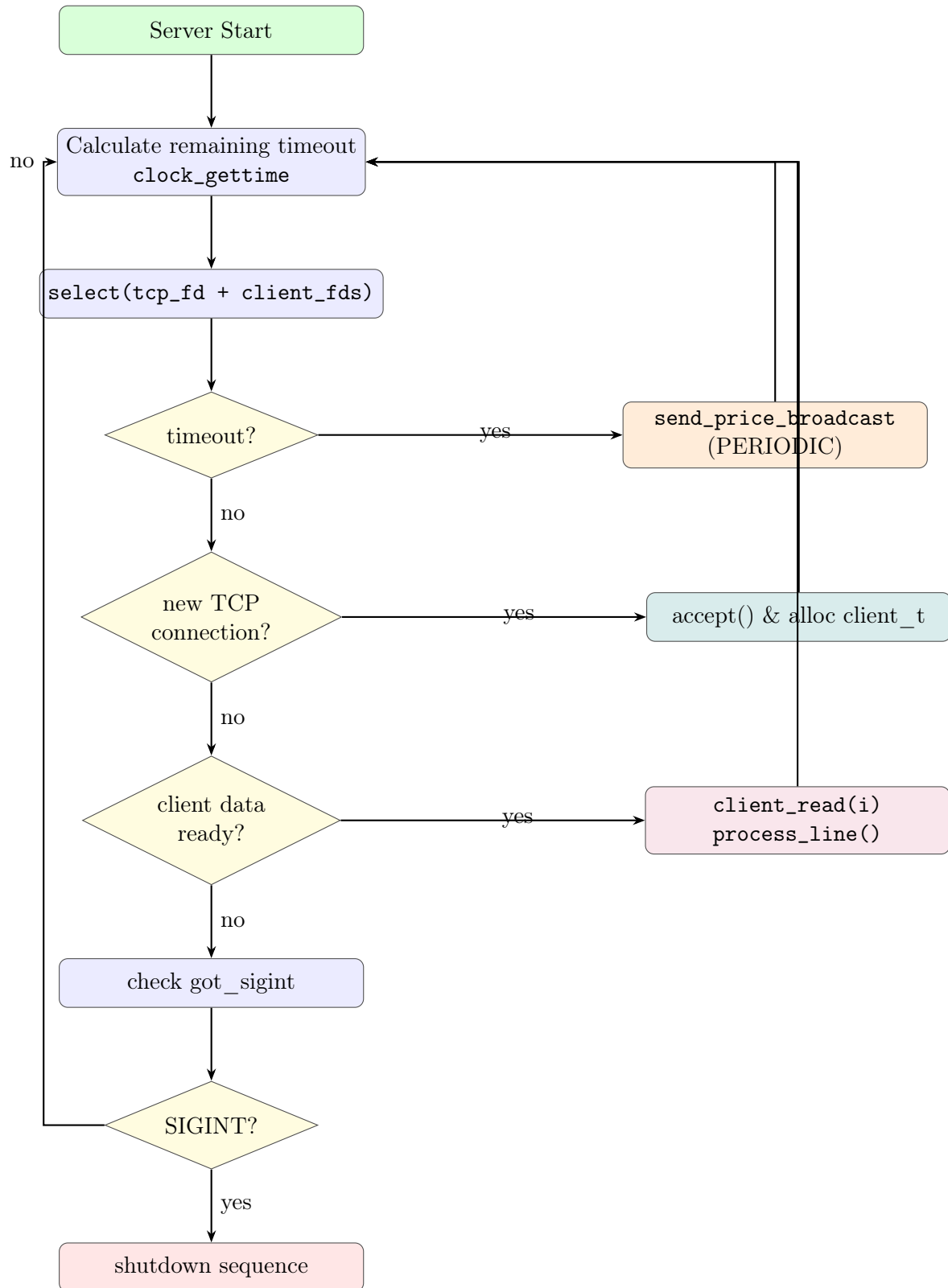
Contents

1	I/O Multiplexing Design	2
1.1	Overview	2
1.2	select() Loop Architecture	3
1.3	5-Second Periodic Broadcast Without sleep()	4
2	Per-Client State	4
2.1	client_t Structure	4
2.2	Portfolio Management	4
2.3	State Cleanup on Disconnect	5
3	Price Update Mechanism	5
3.1	Price Formula	5
3.2	Broadcast Trigger	5
4	Partial Read/Write Handling	6
4.1	The Problem	6
4.2	Line Buffer Solution	6
4.3	Partial Write Handling	6
5	Signal Handling and Shutdown	7
5.1	Server Ctrl+C Sequence	7
5.2	Client Ctrl+D Sequence	7
6	Test Scenarios	8
6.1	Scenario 1: Two Traders Sequential BUY	8
6.2	Scenario 2: 3 Traders + 2 Analysts + 2 Tickers	10
6.3	Scenario 3: ERR INSUFFICIENT_SHARES	15
6.4	Scenario 4: 3 BUY Orders + REPORT + LIST + Partial Read	16
6.5	Scenario 5: kill -9 Hangup and Username Reuse	19
6.6	Scenario 6: SIGINT, Restart, Valgrind	22
7	Challenges and Solutions	25
7.1	Challenge 1: Periodic Broadcast Without sleep()	25
7.2	Challenge 2: Partial Read Handling for Long Messages	25

I/O Multiplexing Design

Overview

The server manages all TCP client connections and the UDP socket within a single thread using the `select()` system call. This design strictly complies with the project requirement of no thread-per-client architecture. A single `select()` loop monitors the TCP listening socket, all connected client file descriptors, and calculates the timeout for the 5-second periodic UDP broadcast.

select() Loop ArchitectureFigure 1: Server main `select()` loop flow

5-Second Periodic Broadcast Without sleep()

The 5-second periodic broadcast is implemented without `sleep()` or any additional thread. Before each `select()` call, the remaining time until the next broadcast is calculated using `clock_gettime(CLOCK_MONOTONIC)`:

Listing 1: Timeout calculation before each `select()` call

```

1 struct timespec now;
2 clock_gettime(CLOCK_MONOTONIC, &now);
3 long elapsed_ms = (now.tv_sec - last_bcast.tv_sec) * 1000
4                 + (now.tv_nsec - last_bcast.tv_nsec) / 1000000;
5 long remain_ms = BCAST_INTERVAL * 1000 - elapsed_ms;
6 if (remain_ms < 0) remain_ms = 0;
7
8 struct timeval tv;
9 tv.tv_sec = remain_ms / 1000;
10 tv.tv_usec = (remain_ms % 1000) * 1000;
11
12 select(maxfd + 1, &rdfs, NULL, NULL, &tv);

```

When `select()` returns 0 (timeout) or when the elapsed time exceeds 5 seconds, `send_price_broadcast` is called and `last_bcast` is reset.

Per-Client State

client_t Structure

Each connected TCP client is represented by a `client_t` struct:

Listing 2: `client_t` structure

```

1 typedef struct {
2     char    symbol[SYMBOL_LEN];
3     int     qty;
4 } holding_t;
5
6 typedef struct {
7     int     fd;
8     char    username[32];
9     char    type[16];           /* "TRADER" or "ANALYST" */
10    char    line_buf[512];       /* partial read buffer */
11    int     buf_len;
12    int     joined;              /* JOIN received? */
13    holding_t portfolio[MAX_STOCKS];
14    int     port_count;
15 } client_t;

```

Portfolio Management

Portfolio tracking is implemented as a dynamic array of `holding_t` entries, one per unique symbol. On a successful BUY, if the symbol already exists in the portfolio, its quantity is incremented; otherwise a new entry is appended. On a successful SELL, the quantity is decremented, and if it reaches zero, the entry is removed by swapping it with the last element ($O(1)$ deletion).

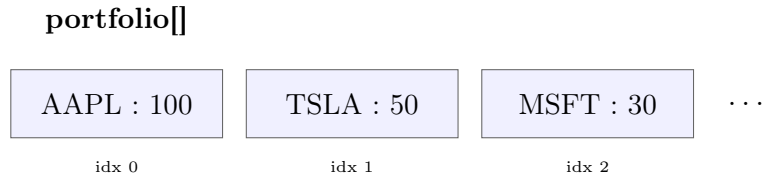


Figure 2: Portfolio array structure for a TRADER client

State Cleanup on Disconnect

When a client disconnects (either via QUIT or hangup), the `disconnect_client()` function resets all fields: `fd = -1`, username cleared, `buf_len = 0`, `port_count = 0`. The slot becomes available for a new connection. This ensures that a killed client's username can be reused immediately.

Price Update Mechanism

Price Formula

Stock prices are updated according to the following rules:

Order	Formula	Constraint
BUY <i>symbol qty</i>	$\text{new_price} = \text{old_price} + \text{qty} \times 0.01$	—
SELL <i>symbol qty</i>	$\text{new_price} = \text{old_price} - \text{qty} \times 0.01$	$\text{new_price} \geq 0.01$

Broadcast Trigger

After every successful BUY or SELL, `send_price_broadcast("TRADE")` is called immediately. If the SELL fails with `ERR_INSUFFICIENT_SHARES`, no broadcast is sent and the price remains unchanged. The server also broadcasts every 5 seconds regardless of trading activity (`trigger=PERIODIC`).

Listing 3: UDP broadcast function

```

1 static void send_price_broadcast(const char *trigger)
2 {
3     char buf[4096];
4     int off = snprintf(buf, sizeof(buf), "PRICE_UPDATE ");
5     for (int i = 0; i < stock_count; i++) {
6         if (i > 0) { buf[off] = ' '; off++; }
7         off += snprintf(buf + off, sizeof(buf) - off,
8                         "%s:%.2f", stocks[i].symbol, stocks[i].price);
9     }
10    buf[off++] = '\n';
11    sendto(udp_fd_g, buf, off, 0,
12           (struct sockaddr *)&bcast_addr_g, sizeof(bcast_addr_g));
13    server_log("[SERVER] PRICE_BROADCAST trigger=%s\n", trigger);
14 }

```

Partial Read/Write Handling

The Problem

TCP is a stream protocol – it does not preserve message boundaries. A single `recv()` call may return only part of a line (e.g., "BUY AAPL 100" instead of "BUY AAPL 100\n"), or it may return multiple lines concatenated. The server must handle both cases correctly.

Line Buffer Solution

Each client maintains a `line_buf[512]` and a `buf_len` counter. Incoming bytes are appended to the buffer, and the buffer is scanned for `'\n'` characters. Each complete line is extracted and passed to `process_line()`. The remaining incomplete bytes stay in the buffer for the next `recv()` call.

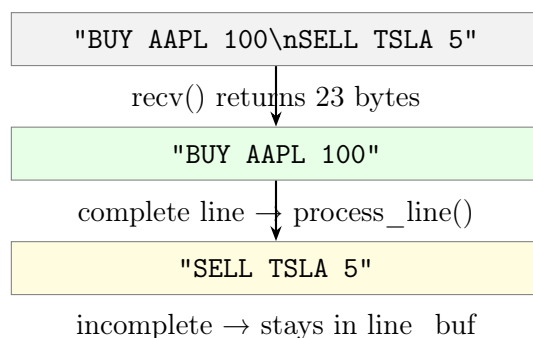


Figure 3: Partial read handling: one `recv()` returns two lines, second is incomplete

Listing 4: `client_read()` – partial read handling

```

1 ssize_t n = recv(c->fd, c->line_buf + c->buf_len, space, 0);
2 if (n <= 0) { disconnect_client(idx, "hangup"); return; }
3 c->buf_len += (int)n;
4
5 while (1) {
6     char *nl = memchr(c->line_buf, '\n', c->buf_len);
7     if (!nl) break; /* no complete line yet */
8     *nl = '\0';
9     process_line(idx, c->line_buf); /* handle complete line */
10
11     int consumed = (int)(nl - c->line_buf) + 1;
12     c->buf_len -= consumed;
13     if (c->buf_len > 0)
14         memmove(c->line_buf, c->line_buf + consumed, c->buf_len);
15 }

```

Partial Write Handling

The `safe_send()` function wraps `send()` in a loop to guarantee that all bytes are transmitted, handling the case where the kernel buffer is full and only part of the message is sent:

Listing 5: `safe_send()` – partial write protection

```
1 static int safe_send(int fd, const char *msg)
2 {
3     size_t total = strlen(msg), sent = 0;
4     while (sent < total) {
5         ssize_t n = send(fd, msg + sent, total - sent, 0);
6         if (n <= 0) return -1;
7         sent += (size_t)n;
8     }
9     return 0;
10 }
```

Signal Handling and Shutdown

Server Ctrl+C Sequence

SIGINT is handled using a volatile `sig_atomic_t` flag. The signal handler only sets this flag – no other operations are performed inside the handler to ensure async-signal safety:

Listing 6: SIGINT handler

```
1 static volatile sig_atomic_t got_sigint = 0;
2 static void handle_sigint(int sig) { (void)sig; got_sigint = 1; }
```

The main `select()` loop checks this flag on every iteration. When set, the server:

1. Exits the `while(!got_sigint)` loop
2. Logs `[SERVER] SHUTDOWN signal=SIGINT`
3. Sends `"SERVER SHUTDOWN\n"` to all connected TCP clients
4. Closes all TCP and UDP file descriptors
5. Logs `[SERVER] CLEANUP_DONE clients=0`
6. Exits

Client Ctrl+D Sequence

When the user presses Ctrl+D on a trader or analyst terminal:

1. `fgets()` returns NULL (EOF on stdin)
2. Client sends `"QUIT\n"` to the server
3. Server responds with `"OK QUIT\n"` and logs `CLIENT_DISCONNECTED username=X reason=QUIT`
4. Client logs `DISCONNECTED reason=QUIT` and closes the socket

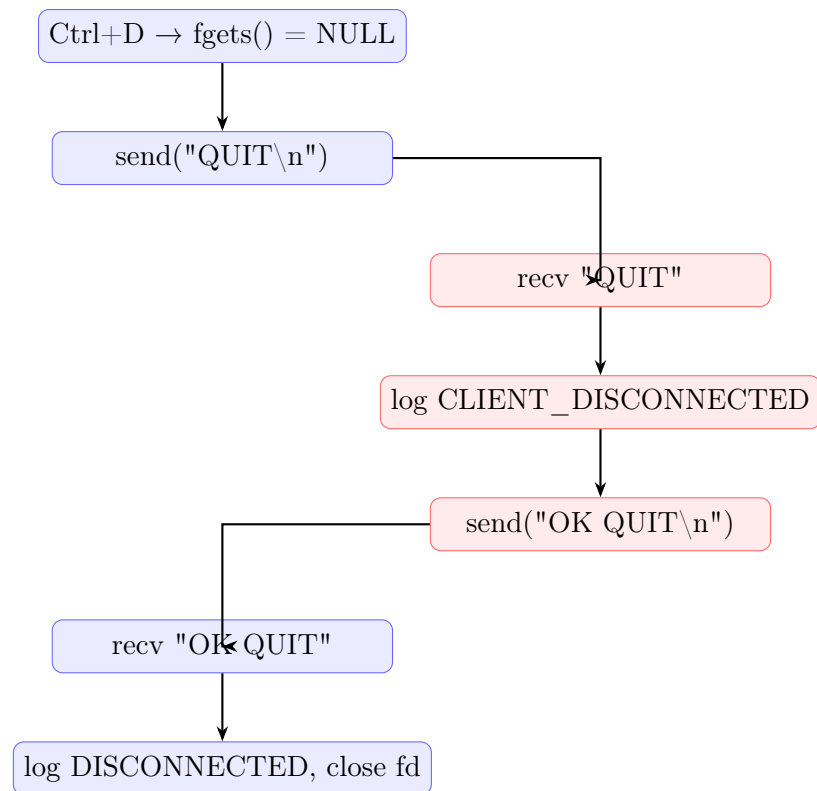


Figure 4: Client Ctrl+D shutdown sequence

Test Scenarios

Scenario 1: Two Traders Sequential BUY

Two traders (alice, bob) connect. Alice sends BUY AAPL 100, bob sends BUY AAPL 50. Both check their portfolio and press Ctrl+D.

Expected: alice PORTFOLIO = AAPL:100, bob PORTFOLIO = AAPL:50. Final AAPL price = $150.00 + 1.00 + 0.50 = \mathbf{151.50}$.

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./ServerTrd -p 8080 -u 8081 -s stocks.txt -l server.log
[SERVER] SERVER START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=AAPL qty=50 old_price=151.00 new_price=151.50
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
^C[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 5: Scenario 1 – Server Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
PORTFOLIO
[TRADER alice] SENT PORTFOLIO
[TRADER alice] RECEIVED OK PORTFOLIO AAPL:100
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 6: Scenario 1 – alice Trader Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 bob
[TRADER bob] CONNECTED server=127.0.0.1:8080
[TRADER bob] SENT JOIN
[TRADER bob] RECEIVED OK JOIN bob
BUY AAPL 50
[TRADER bob] SENT BUY AAPL 50
[TRADER bob] RECEIVED OK BUY AAPL 50 151.50
PORTFOLIO
[TRADER bob] SENT PORTFOLIO
[TRADER bob] RECEIVED OK PORTFOLIO AAPL:50
[TRADER bob] SENT QUIT
[TRADER bob] RECEIVED OK QUIT
[TRADER bob] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 7: Scenario 1 – bob Trader Terminal

```

feYZa@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=AAPL qty=50 old_price=151.00 new_price=151.50
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
feYZa@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 8: Scenario 1 – server.log Output

Scenario 2: 3 Traders + 2 Analysts + 2 Tickers

Three traders and two analysts connect simultaneously along with two ticker clients. Traders send BUY and SELL orders on different stocks. Analyst1 sends LIST, analyst2 sends REPORT.

Expected: LIST contains all 5 usernames. REPORT contains all 30 stocks with updated prices. Both tickers receive PRICE_UPDATE after every trade.

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./ServerTrd -p 8080 -u 8081 -s stocks.txt -l server.log
[SERVER] SERVER START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=7
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] JOIN username=charlie type=TRADER fd=8
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=9 ip=127.0.0.1
[SERVER] JOIN username=analyst1 type=ANALYST fd=9
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=10 ip=127.0.0.1
[SERVER] JOIN username=analyst2 type=ANALYST fd=10
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=MSFT qty=50 old_price=250.00 new_price=250.50
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=MSFT qty=200 old_price=380.00 new_price=382.00
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] SELL trader=bob symbol=MSFT qty=100 old_price=382.00 new_price=381.00
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=charlie symbol=GOOGL qty=10 old_price=2800.00 new_price=2800.10
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=analyst2 reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=analyst1 reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=charlie reason=QUIT
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
^C[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 9: Scenario 2 – Server Terminal

```

feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$ ./trader 8080
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
BUY TSLA 50
[TRADER alice] SENT BUY TSLA 50
[TRADER alice] RECEIVED OK BUY TSLA 50 250.50
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$

```

Figure 10: Scenario 2 – Ticker 1 Terminal

```

feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$ ./trader 8080
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
BUY TSLA 50
[TRADER alice] SENT BUY TSLA 50
[TRADER alice] RECEIVED OK BUY TSLA 50 250.50
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$

```

Figure 11: Scenario 2 – Ticker 2 Terminal

```

feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
BUY TSLA 50
[TRADER alice] SENT BUY TSLA 50
[TRADER alice] RECEIVED OK BUY TSLA 50 250.50
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyz@debian11:~/Desktop/GTU-CSE344-System-Programming/Finals$

```

Figure 12: Scenario 2 – alice Trader Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 bob
[TRADER bob] CONNECTED server=127.0.0.1:8080
[TRADER bob] SENT JOIN
[TRADER bob] RECEIVED OK JOIN bob
BUY MSFT 200
[TRADER bob] SENT BUY MSFT 200
[TRADER bob] RECEIVED OK BUY MSFT 200 382.00
SELL MSFT 100
[TRADER bob] SENT SELL MSFT 100
[TRADER bob] RECEIVED OK SELL MSFT 100 381.00
[TRADER bob] SENT QUIT
[TRADER bob] RECEIVED OK QUIT
[TRADER bob] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 13: Scenario 2 – bob Trader Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 charlie
[TRADER charlie] CONNECTED server=127.0.0.1:8080
[TRADER charlie] SENT JOIN
[TRADER charlie] RECEIVED OK JOIN charlie
BUY GOOGL 10
[TRADER charlie] SENT BUY GOOGL 10
[TRADER charlie] RECEIVED OK BUY GOOGL 10 2800.10
[TRADER charlie] SENT QUIT
[TRADER charlie] RECEIVED OK QUIT
[TRADER charlie] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 14: Scenario 2 – charlie Trader Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./analyst 127.0.0.1 8080 analyst1
[ANALYST analyst1] CONNECTED server=127.0.0.1:8080
[ANALYST analyst1] SENT JOIN
[ANALYST analyst1] RECEIVED OK JOIN analyst1
LIST
[ANALYST analyst1] SENT LIST
[ANALYST analyst1] RECEIVED OK LIST alice,bob,charlie,analyst1,analyst2
[ANALYST analyst1] SENT QUIT
[ANALYST analyst1] RECEIVED OK QUIT
[ANALYST analyst1] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 15: Scenario 2 – analyst1 Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./analyst 127.0.0.1 8080 analyst2
[ANALYST analyst2] CONNECTED server=127.0.0.1:8080
[ANALYST analyst2] SENT JOIN
[ANALYST analyst2] RECEIVED OK JOIN analyst2
REPORT
[ANALYST analyst2] SENT REPORT
[ANALYST analyst2] RECEIVED OK REPORT AAPL:151.00,GOOGL:2800.10,MSFT:381.00,TSLA:250.50,AMZN:3400.00,NVDA:450.00,META:300.00,NFLX:500.00,AMD:120.00,INTC:35.00,ORCL:110.00,IBM:140.00,CSCO:55.00,COM:130.00,AVGO:600.00,TXN:170.00,MU:80.00,WDC:50.00,AMAT:140.00,LRCX:650.00,KLAC:450.00,SNPS:380.00,CDNS:200.00,MVRL:60.00,ON:70.00,MPWR:350.00,SNKS:110.00,QRVO:100.00,WOLF:30.00,SITM:150.00
[ANALYST analyst2] SENT QUIT
[ANALYST analyst2] RECEIVED OK QUIT
[ANALYST analyst2] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 16: Scenario 2 – analyst2 Terminal


```

feYZa@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=7
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] JOIN username=charlie type=TRADER fd=8
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=9 ip=127.0.0.1
[SERVER] JOIN username=analyst1 type=ANALYST fd=9
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=10 ip=127.0.0.1
[SERVER] JOIN username=analyst2 type=ANALYST fd=10
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=TSLA qty=50 old_price=250.00 new_price=250.50
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=MSFT qty=200 old_price=380.00 new_price=382.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SELL trader=bob symbol=MSFT qty=100 old_price=382.00 new_price=381.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=charlie symbol=GOOGL qty=10 old_price=2800.00 new_price=2800.10
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=analyst2 reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=analyst1 reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=charlie reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP DONE clients=0
feYZa@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 17: Scenario 14 – server.log Output

Scenario 3: ERR INSUFFICIENT_SHARES

One trader connects and buys 100 shares of AAPL, then attempts to sell 999 shares. An analyst queries the price of AAPL.

Expected: SELL returns ERR INSUFFICIENT_SHARES. No UDP broadcast after failed SELL. Analyst sees price updated only by the successful BUY (151.00).

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./ServerTrd -p 8080 -u 8081 -s stocks.txt -l server.log
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=carol type=ANALYST fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=carol reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
^C[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 18: Scenario 3 – Server Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
SELL AAPL 999
[TRADER alice] SENT SELL AAPL 999
[TRADER alice] RECEIVED ERR INSUFFICIENT_SHARES
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 19: Scenario 3 – alice Trader Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./analyst 127.0.0.1 8080 carol
[ANALYST carol] CONNECTED server=127.0.0.1:8080
[ANALYST carol] SENT JOIN
[ANALYST carol] RECEIVED OK JOIN carol
PRICE AAPL
[ANALYST carol] SENT PRICE AAPL
[ANALYST carol] RECEIVED OK PRICE AAPL 151.00
[ANALYST carol] SENT QUIT
[ANALYST carol] RECEIVED OK QUIT
[ANALYST carol] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 20: Scenario 3 – carol Trader Terminal


```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=carol type=ANALYST fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=carol reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 21: Scenario 3 – server.log Output

Scenario 4: 3 BUY Orders + REPORT + LIST + Partial Read

One trader buys three different stocks. An analyst sends REPORT then LIST. One ticker monitors broadcasts.

Expected: Ticker receives exactly 3 PRICE_UPDATE messages. REPORT includes all 30 stocks with 3 updated prices. LIST shows both connected usernames. Long REPORT message verifies partial read handling.

Figure 22: Scenario 4 – Server Terminal

Figure 23: Scenario 4 – Ticker Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
BUY TSLA 50
[TRADER alice] SENT BUY TSLA 50
[TRADER alice] RECEIVED OK BUY TSLA 50 250.50
BUY MSFT 200
[TRADER alice] SENT BUY MSFT 200
[TRADER alice] RECEIVED OK BUY MSFT 200 382.00
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █
```

Figure 24: Scenario 4 – alice Trader Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./analyst 127.0.0.1 8080 carol
[ANALYST carol] CONNECTED server=127.0.0.1:8080
[ANALYST carol] SENT JOIN
[ANALYST carol] RECEIVED OK JOIN carol
REPORT
[ANALYST carol] SENT REPORT
[ANALYST carol] RECEIVED OK REPORT AAPL:151.00,GOOGL:2800.00,MSFT:382.00,TSLA:250.50,AMZN:3400.00,NVDA:450.00,META:300.00,NFLX:500.00,AMD:120.00,INTC:35.00,ORCL:110.00,IBM:140.00,CSCO:55.00,QCOM:130.00,AVGO:600.00,TXN:170.00,MU:80.00,WDC:50.00,AMAT:140.00,LRCX:650.00,KLAC:450.00,SNPS:380.00,CDNS:200.00,MRVL:60.00,MPWR:350.00,SWKS:110.00,QRVO:100.00,WOLF:30.00,SITM:150.00
LIST
[ANALYST carol] SENT LIST
[ANALYST carol] RECEIVED OK LIST alice,carol
[ANALYST carol] SENT QUIT
[ANALYST carol] RECEIVED OK QUIT
[ANALYST carol] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █
```

Figure 25: Scenario 4 – carol Analyst Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=TESLA qty=50 old_price=250.00 new_price=250.50
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=MSFT qty=200 old_price=380.00 new_price=382.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[SERVER] JOIN username=carol type=ANALYST fd=7
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=carol reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 26: Scenario 4 – server.log Output

Scenario 5: kill -9 Hangup and Username Reuse

Three traders connect. Trader1 (alice) is killed with `kill -9`. Immediately a new trader connects with the same username. Trader2 (bob) and Trader3 continue unaffected.

Expected: Server detects hangup and logs `reason=hangup`. New trader with same username is accepted. Other traders are unaffected.

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./ServerTrd -p 8080 -u 8081 -s stocks.txt -l server.log
[SERVER] SERVER START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=10 old_price=150.00 new_price=150.10
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=7
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=MSFT qty=5 old_price=380.00 new_price=380.05
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=hangup
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=10 old_price=150.10 new_price=150.20
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=TESLA qty=20 old_price=250.00 new_price=250.20
[SERVER] PRICE BROADCAST trigger=TRADE
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] PRICE BROADCAST trigger=PERIODIC
[SERVER] PRICE BROADCAST trigger=PERIODIC
^C[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ █

```

Figure 27: Scenario 5 – Server Terminal

```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 10
[TRADER alice] SENT BUY AAPL 10
[TRADER alice] RECEIVED OK BUY AAPL 10 150.10
Killed
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 28: Scenario 5 – alice (killed) Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 bob
[TRADER bob] CONNECTED server=127.0.0.1:8080
[TRADER bob] SENT JOIN
[TRADER bob] RECEIVED OK JOIN bob
BUY MSFT 5
[TRADER bob] SENT BUY MSFT 5
[TRADER bob] RECEIVED OK BUY MSFT 5 380.05
BUY TSLA 20
[TRADER bob] SENT BUY TSLA 20
[TRADER bob] RECEIVED OK BUY TSLA 20 250.20
[TRADER bob] SENT QUIT
[TRADER bob] RECEIVED OK QUIT
[TRADER bob] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$
```

Figure 29: Scenario 5 – bob Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ pgrep -a trader
2493 ./trader 127.0.0.1 8080 alice
2496 ./trader 127.0.0.1 8080 bob
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ kill -9 2493
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$
```

Figure 30: Scenario 5 – kill Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 10
[TRADER alice] SENT BUY AAPL 10
[TRADER alice] RECEIVED OK BUY AAPL 10 150.20
[TRADER alice] SENT QUIT
[TRADER alice] RECEIVED OK QUIT
[TRADER alice] DISCONNECTED reason=QUIT
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$
```

Figure 31: Scenario 5 – new Alice Terminal


```

feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=10 old_price=150.00 new_price=150.10
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[SERVER] JOIN username=bob type=TRADER fd=7
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=MSFT qty=5 old_price=380.00 new_price=380.05
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=alice reason=hangup
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=alice symbol=AAPL qty=10 old_price=150.10 new_price=150.20
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=bob symbol=TSLA qty=20 old_price=250.00 new_price=250.20
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_DISCONNECTED username=bob reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 32: Scenario 5 – server.log Output

Scenario 6: SIGINT, Restart, Valgrind

Server runs under valgrind. Trader sends BUY, then server receives SIGINT (Ctrl+C). Server is immediately restarted on the same ports. A new trader connects and sends BUY.

Expected: All TCP clients receive SERVER SHUTDOWN. CLEANUP_DONE logged. Valgrind reports 0 leaks and 0 errors. Server restarts immediately (SO_REUSEADDR). No zombie processes.

[illegible]

Figure 33: Scenario 6 – Valgrind + Server Terminal

```

fezya@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ ./trader 127.0.0.1 8080 alice
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] RECEIVED OK JOIN alice
BUY AAPL 100
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
[TRADER alice] DISCONNECTED reason=shutdown
fezya@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$

```

Figure 34: Scenario 6 – alice Trader Terminal

Figure 35: Scenario 6 – Ticker Terminal

Figure 36: Scenario 6 – newtrader Terminal

Figure 37: Scenario 6 – Zombie Check Terminal

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$ cat server.log
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=30
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] JOIN username=newtrader type=TRADER fd=6
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] BUY trader=newtrader symbol=MSFT qty=50 old_price=380.00 new_price=380.50
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] CLIENT_DISCONNECTED username=newtrader reason=QUIT
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] PRICE_BROADCAST trigger=PERIODIC
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/Final$
```

Figure 38: Scenario 6 – server.log Output

Challenges and Solutions

Challenge 1: Periodic Broadcast Without sleep()

Problem: The 5-second periodic broadcast must not block the `select()` loop or use `sleep()`, which would prevent the server from responding to client events during the wait period. Using a separate thread was also forbidden.

Solution: The `select()` timeout parameter was used as the timer. Before each `select()` call, the remaining time until the next broadcast is computed using `clock_gettime(CLOCK_MONOTONIC)`, which is immune to system clock adjustments. If `select()` returns 0 (timeout expired), the broadcast is sent and the reference timestamp is reset. This approach guarantees both timely broadcasts and immediate responsiveness to client events.

Challenge 2: Partial Read Handling for Long Messages

Problem: TCP is a stream protocol and does not preserve message boundaries. The REPORT response, which contains all 30 stocks formatted as `OK REPORT symbol:price,...`, can exceed typical MTU sizes. Without proper buffering, `recv()` on the client side might

receive the response in multiple fragments, causing the client to process an incomplete line.

Solution: Both server and client implement a `line_buf` accumulator. Incoming bytes are appended to the buffer and scanned for `'\n'`. Only complete lines are passed to the command processor; partial lines remain in the buffer until the next `recv()` delivers the rest. On the server side, `safe_send()` ensures all bytes of a response are transmitted even when the socket send buffer is temporarily full.